UNIT II:-Software Quality Assurance

2.1 Software Quality Assurance:-Software Quality Assurance (SQA) is simply a way to assure quality in the software. It is the set of activities that ensure processes, procedures as well as standards are suitable for the project and implemented correctly. It is a process that works parallel to Software Development. It focuses on improving the process of development of software so that problems can be prevented before they become major issues.

Software Quality Assurance is a kind of **Umbrella activity** that is applied throughout the software process.

Quality

Quality in a product or service can be defined by several measurable characteristics. Each of these characteristics plays a crucial role in determining the overall quality.

Generally, the quality of the software is verified by third-party organizations like international standard organizations .

Elements of Software Quality Assurance (SQA)

- **Standards:** The IEEE, ISO, and other standards organizations have produced a broad array of software engineering standards and related documents. The job of SQA is to ensure that standards that have been adopted are followed and that all work products conform to them.
- **Reviews and audits:** Technical reviews are a quality control activity performed by software engineers for software engineers. Their intent is to uncover errors. Audits are a type of review performed by SQA personnel (people employed in an organization) with the intent of ensuring that quality guidelines are being followed for software engineering work.
- **Testing:** Software testing is a quality control function that has one primary goal to find errors. The job of SQA is to ensure that testing is properly planned and efficiently conducted for primary goal of software.
- **Error/defect collection and analysis** : SQA collects and analyzes error and defect data to better understand how errors are introduced and what software engineering activities are best suited to eliminating them.
- **Change management:** SQA ensures that adequate change management practices have been instituted.

- **Education:** Every software organization wants to improve its software engineering practices. A key contributor to improvement is education of software engineers, their managers, and other stakeholders. The SQA organization takes the lead in software process improvement which is key proponent and sponsor of educational programs.
- **Security management:** SQA ensures that appropriate process and technology are used to achieve software security.
- **Safety:** SQA may be responsible for assessing the impact of software failure and for initiating those steps required to reduce risk.
- **Risk management:** The SQA organization ensures that risk management activities are properly conducted and that risk-related contingency plans have been established.

Focus of Software Quality Assurance (SQA)

The Software Quality Assurance (SQA) focuses on the following :
Software Quality Assurance (SQA)

- **Software's Portability:** Software's portability refers to its ability to be easily transferred or adapted to different environments or platforms without needing significant modifications. This ensures that the software can run efficiently across various systems, enhancing its accessibility and flexibility.
- **Software's Usability:** Usability of software refers to how easy and intuitive it is for users to interact with and navigate through the application. A high level of usability ensures that users can effectively accomplish their tasks with minimal confusion or frustration, leading to a positive user experience.

- **Software's Reusability:** Reusability in software development involves designing components or modules that can be reused in multiple parts of the software or in different projects. This promotes efficiency and reduces development time by eliminating the need to reinvent the wheel for similar functionalities, enhancing productivity and maintainability.
- **Software's Correctness:** Correctness of software refers to its ability to produce the desired results under specific conditions or inputs. Correct software behaves as expected without errors or unexpected behaviors, meeting the requirements and specifications defined for its functionality.
- **Software's Maintainability:** Maintainability of software refers to how easily it can be modified, updated, or extended over time. Well maintained software is structured and documented in a way that allows developers to make changes efficiently without introducing errors or compromising its stability.
- **Software's Error Control:** Error control in software involves implementing mechanisms to detect, handle, and recover from errors or unexpected situations gracefully. Effective error control ensures that the software remains robust and reliable, minimizing disruptions to users and providing a smoother experience overall.

Major Activities in Software Quality Assurance (SQA)

- **SQA Management Plan:** Make a plan for how you will carry out the SQA throughout the project. Think about which set of software engineering activities are the best for project. Check level of SQA team skills.
- **Set The Check Points:** SQA team should set checkpoints. Evaluate the performance of the project on the basis of collected data on different check points.
- **Measure Change Impact:** The changes for making the correction of an error sometimes re introduces more errors keep the measure of impact of change on project. Reset the new change to check the compatibility of this fix with whole project.
- **Multi testing Strategy:** Do not depend on a single testing approach. When you have a lot of testing approaches available use them.
- **Manage Good Relations:** In the working environment managing good relations with other teams involved in the project development is mandatory. Bad relation of SQA team with programmer’s team will impact directly and badly on project.

- **Maintaining records and reports:** Comprehensively document and share all QA records, including test cases, defects, changes, and cycles, for stakeholder awareness and future reference.
- **Reviews software engineering activities:** The SQA group identifies and documents the processes. The group also verifies the correctness of software product.
- **Formalize deviation handling:** Track and document software deviations meticulously. Follows established procedures for handling variances.

Benefits of Software Quality Assurance (SQA)

- SQA produces high quality software.
- High quality application saves time and cost.
- SQA is beneficial for better reliability.
- SQA is beneficial in the condition of no maintenance for a long time.
- High quality commercial software increase market share of company.
- Improving the process of creating software.
- Improves the quality of the software.

2.2 Software Review and Its types

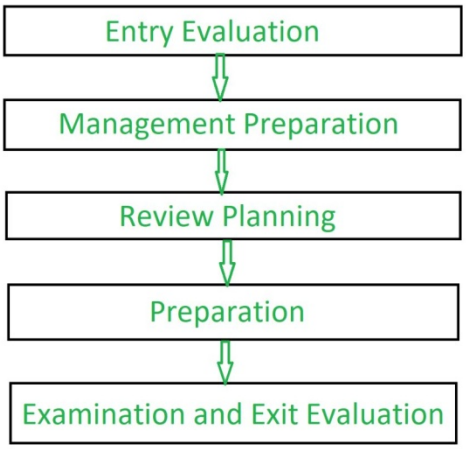
Software Review is a systematic inspection of software by one or more individuals who work together to find and resolve errors and defects in the software during the early stages of the Software Development Life Cycle (SDLC). A software review is an essential part of the Software Development Life Cycle (SDLC) that helps software engineers in validating the quality, functionality, and other vital features and components of the software. It is a whole process that includes testing the software product and it makes sure that it meets the requirements stated by the client.
Usually performed manually, software review is used to verify various documents like requirements, system designs, codes, test plans, and test cases.

Objectives of Software Review

The objective of the software review is:

1. To improve the productivity of the development team.
2. To make the testing process time and cost-effective.
3. To make the final software with fewer defects.
4. To eliminate the inadequacies.

Process of Software Review



1. **Entry Evaluation:** By confirming documentation, fulfilling entry requirements and assessing stakeholder and team preparation, you can determine the software's availability.
2. **Management Preparation**: To get ready for the review process, assign roles, gather resources and provide brief management.
3. **Review Planning:** Establish the review's goals and scope, invite relevant parties and set a time for the meeting.
4. **Preparation**: Distribute appropriate resources, give reviewers time to get familiar and promote issue identification to help them prepare.
5. **Examination and Exit Evaluation:** Reviewers should collaborate to examine the results, record concerns, and encourage candid communication in meetings. It assess the results, make remedial plans based on flaws that have been reported and assess the process's overall efficacy.

Types of Software Reviews

There are mainly 3 types of software reviews:

1. Software Peer Review

Peer review is the process of assessing the technical content and quality of the product and it is usually conducted by the author of the work product along with some other developers.
Peer review is performed in order to examine or resolve the defects in the software, whose quality is also checked by other members of the team.
Peer Review has following types:

1. **Code Review:** Computer source code is examined in a systematic way.
2. **Pair Programming:** It is a code review where two developers develop code together at the same platform.
3. **Walkthrough:** Members of the development team is guided by author and other interested parties and the participants ask questions and make comments about defects.
4. **Technical Review:** A team of highly qualified individuals examines the software product for its client's use and identifies technical defects from specifications and standards.
5. **Inspection:** In inspection the reviewers follow a well-defined process to find defects.

2. Software Management Review

Software Management Review evaluates the work status. In this section decisions regarding downstream activities are taken.

3. Software Audit Review

Software Audit Review is a type of external review in which one or more critics, who are not a part of the development team, organize an independent inspection of the software product and its processes to assess their compliance with stated specifications and standards. This is done by managerial level people.

Advantages of Software Review

1. Defects can be identified earlier stage of development (especially in formal review).

2. Earlier inspection also reduces the maintenance cost of software.
3. It can be used to train technical authors.
4. It can be used to remove process inadequacies that encourage defects.

2.3 FORMAL TECHNICAL REVIEWS:-

A formal technical review is a software quality assurance activity performed by software engineers (and others). The objectives of the FTR are

(1) To uncover errors in function, logic, or implementation for any representation of the software;

(2) To verify that the software under review meets its requirements;

(3) To ensure that the software has been represented according to predefined standards;

(4) To achieve software that is developed in a uniform manner; and

(5) To make projects more manageable. In addition, the FTR serves as a training ground, enabling junior engineers to observe different approaches to software analysis, design, and implementation. The FTR also serves to promote backup and continuity because a number of people become familiar with parts of the software that they may not have otherwise seen

1) The Review Meeting:-

Regardless of the FTR format that is chosen, every review meeting should abide by the following constraints:

• Between three and five people (typically) should be involved in the review.

• Advance preparation should occur but should require no more than two hours of work for each person.

• The duration of the review meeting should be less than two hours.

Given these constraints, it should be obvious that an FTR focuses on a specific (and small) part of the overall software. For example, rather than attempting to review an entire design, walkthroughs are conducted for each component or small group of components. By narrowing focus, the FTR has a higher likelihood of uncovering errors.

At the end of the review, all attendees of the FTR must decide whether to

(1) Accept the product without further modification,

(2) Reject the product due to severe errors (once corrected, another review must be performed), or

(3) Accept the product provisionally (minor errors have been encountered and must be corrected, but no additional review will be required). The decision made, all FTR attendees complete a sign-off, indicating their participation in the review and their concurrence with the review team's finding

2) Review Reporting and Record Keeping:

During the FTR, a reviewer (the recorder) actively records all issues that have been raised. These are summarized at the end of the review meeting and a review issues list is produced. In addition, a formal technical review summary report is completed. A review summary report answers three questions:

1. What was reviewed?

2. Who reviewed it?

3. What were the findings and conclusions?

The review summary report is a single page form (with possible attachments). It becomes part of the project historical record and may be distributed to the

project leader and other interested parties. The review issues list serves two purposes:

(1) to identify problem areas within the product and

(2) to serve as an action item checklist that guides the producer as corrections are made.

An issues list is normally attached to the summary report

3)Review Guidelines

Guidelines for the conduct of formal technical reviews must be established in advance, distributed to all reviewers, agreed upon, and then followed

1)Review the product, not the producer. An FTR involves people and egos. Conducted properly, the FTR should leave all participants with a warm feeling of accomplishment

2) Set an agenda and maintain it. One of the key maladies of meetings of all types is drift. An FTR must be kept on track and on schedule

3)Limit debate and rebuttal. When an issue is raised by a reviewer, there may not be universal agreement on its impact

4)Enunciate problem areas, but don't attempt to solve every problem noted. A review is not a problem-solving session

5) Take written notes. It is sometimes a good idea for the recorder to make notes on a wall board, so that wording and priorities can be assessed by other reviewers as information is recorded.

6) Limit the number of participants and insist upon advance preparation. Two heads are better than one, but 14 are not necessarily better than 4 keep the number of people involved to the necessary minimum.

7) Develop a checklist for each product that is likely to be reviewed. A checklist helps the review leader to structure the FTR meeting and helps each reviewer to focus on important issues.

2.4 SOFTWARE RELIABILITY:-

Software Reliability in software testing refers to the probability that software will function without failure under given conditions for a specified period of time. It's a key aspect of **software quality** and ensures the system performs consistently as expected.

Definition

Software Reliability = the likelihood of the software performing its intended functions correctly without failure over a defined time period in a specific environment.

✅ Key Characteristics of Software Reliability

1. **Fault-Free Operation**: The software should operate without crashes or errors.
2. **Consistency**: Output should be consistent and predictable.
3. **Recoverability**: Ability to recover from failures quickly and accurately.
4. **Availability**: Percentage of time the software is operational and accessible.

Role in Software Testing

Software reliability testing helps ensure that the software:

- Performs correctly under real-world conditions.
- Can handle unexpected inputs and edge cases.
- Maintains functionality over time without degradation.

Types of Reliability Testing

Test Type	Purpose
Regression Testing	Ensures that new changes haven't broken existing functionality.
Stress Testing	Checks how the software behaves under extreme conditions.
Load Testing	Tests how the software performs under expected workload.
Failure Testing	Determines software behavior during failure or error scenarios.
Recovery Testing	Assesses the ability to recover from crashes or hardware failures.

Metrics to Measure Software Reliability

1. **Mean Time Between Failures (MTBF)**
   = Total uptime / Number of failures
   - Higher MTBF → Better reliability.
2. **Mean Time To Failure (MTTF)**
   = Average time the software operates before it fails.
3. **Defect Density**
   = Number of confirmed defects / Size of the software (e.g., LOC).
4. **Availability (%)**
   = (Uptime / (Uptime + Downtime)) × 100

5. **Importance of Software Reliability**

- Reduces risk of failure in critical systems (e.g., medical, aviation).
- Increases user trust and satisfaction.
- Lowers maintenance and support costs.

- Improves market competitiveness.

Example Scenario

A banking app must remain available and accurate 99.999% of the time. Through reliability testing, developers can simulate heavy usage and edge cases (like sudden disconnections), ensuring it behaves correctly and recovers gracefully.

2.5 A Software Quality Assurance Plan:

A **Software Quality Assurance (SQA) Plan** is a critical document in software testing and development. It outlines how a project will ensure that software meets the required quality standards. This plan includes procedures, techniques, tools, and responsibilities used to achieve quality assurance goals throughout the software development lifecycle (SDLC).

Key Components of a Software Quality Assurance (SQA) Plan

1. Purpose and Scope

- **Purpose**: Defines the objectives of the SQA plan.
- **Scope**: Specifies the boundaries of the plan, including which software products or components it covers.

2. References

- Lists documents, standards, and guidelines referenced in the plan (e.g., IEEE 730, ISO 9001, CMMI).

3. Software Quality Objectives

- Describes measurable quality goals such as:
    - Defect density (e.g., <1 per 1,000 LOC)
    - Performance benchmarks
    - Compliance with coding standards

- Reliability and availability targets

4. Roles and Responsibilities

- Details the responsibilities of:
  - **QA Manager**: Oversees QA activities and ensures compliance
  - **Testers/QA Engineers**: Execute tests, report bugs
  - **Developers**: Fix bugs, follow quality practices
  - **Project Manager**: Ensures integration of QA into project planning

5. Standards, Practices, and Conventions

- Lists the development and testing standards to be followed (e.g., naming conventions, coding standards, test documentation formats).

6. SQA Activities

- **Reviews & Audits**: Code reviews, design reviews, internal audits
- **Testing**:
  - Unit testing
  - Integration testing
  - System testing
  - Acceptance testing
- **Defect tracking and management**

7. Tools, Techniques, and Methodologies

- Specifies the tools and frameworks used (e.g., JIRA, Selenium, Jenkins, JUnit).

- QA methodologies (Agile, Waterfall, DevOps testing, etc.)

8. Risk Management

- Identifies potential risks to quality (e.g., tight deadlines, scope creep).
- Describes mitigation strategies (e.g., buffer time, automated testing).

9. Metrics and Reporting

- Defines the metrics to monitor and evaluate quality:
    - Test coverage
    - Number of defects found
    - Mean time to failure (MTTF)
    - Defect leakage rate
- Reporting frequency (e.g., weekly QA reports)

10. Training and Certification

- Describes training plans for the QA team on tools, procedures, and best practices.

11. Review and Approval

- Details the process for reviewing and approving the SQA plan, including sign-offs from stakeholders.

✅ Benefits of an SQA Plan

- Ensures consistent quality practices
- Reduces defects and rework
- Enhances customer satisfaction

- Supports process improvement and compliance